

Dossier de funcionalidades originales — ComprameLaFoto

Obra: ComprameLaFoto (plataforma integrante de DNX Suite) **Tipo:** Programa de computación — obra inédita **Titular:** [COMPLETAR: nombre, CUIT, nacionalidad, domicilio, % de titularidad de cada autor] **Fecha del snapshot:** [COMPLETAR al generar el paquete] **Destino:** Depósito en custodia de obra inédita — Dirección Nacional del Derecho de Autor (DNDA) **Versión del documento:** 1.0 — 3 de septiembre de 2026

0. Alcance y límites de esta protección

Este documento no convierte una funcionalidad en monopolio, y conviene que quede escrito acá para que nadie lo lea esperando otra cosa.

En Argentina, la Ley 24.481 (art. 6, inc. c) excluye expresamente de la noción de invención a "los planes, reglas y métodos para el ejercicio de actividades intelectuales... o para actividades económico-comerciales, así como los programas de computación". Un tercero puede construir su propio álbum que se revela con una selfie y no estará infringiendo nada por el solo hecho de hacerlo.

Lo que sí queda protegido, y es lo que este dossier construye:

Capa	Qué protege	Vía
Expresión	El código fuente concreto, la arquitectura de datos, la secuencia exacta de estados, los textos de interfaz	Ley 11.723 (arts. 1 y 55 bis) — depósito DNDA
Identidad	El nombre de la plataforma y el nombre de cada funcionalidad	Ley 22.362 — registro de marca en INPI
Parámetros	Umbrales, tiempos, políticas de retención que no viajan al navegador	Ley 24.766 — secreto comercial + NDAs
Anterioridad	La fecha cierta de que todo esto existía antes que la copia	Depósito DNDA (fecha de ingreso)

El valor práctico del depósito es probatorio: frente a una copia sustancial —código reutilizado, misma secuencia de estados, mismos textos, misma estructura de tablas— este documento fechado es lo que permite sostener autoría y anterioridad. Cuanto más específico sea, más sirve. Por eso cada ficha registra no solo *qué hace* la funcionalidad, sino *qué decisión concreta se tomó* donde había varias posibles.

Nota: este dossier es documentación técnica preparada para un trámite. No constituye asesoramiento legal. La estrategia final debería revisarla un agente de la propiedad industrial matriculado.

1. Identificación de la obra

Campo	Valor
Denominación	ComprameLaFoto
Suite	DNX Suite (monorepo Turborepo + pnpm workspaces)
Aplicaciones hermanas	FotoRank, Info Spot, Fotoffice, Clickaton
Naturaleza	Plataforma web de venta de fotografía de eventos, escolar y deportiva
Stack	Next.js 16 (App Router), TypeScript, PostgreSQL + Prisma, Cloudflare R2, AWS Rekognition, Sharp, MercadoPago, Resend
Escala del código	455 modelos de datos; ~751 rutas y endpoints en la app; ~834 módulos de biblioteca
Ruta en repositorio	<code>dnx-suite/apps/compramelafoto</code> + <code>dnx-suite/packages/db</code>

Titularidad — a completar antes del depósito

El formulario de la DNDA exige, por cada autor: nombre y apellido, CUIT/CUIL, nacionalidad, domicilio y **porcentaje de titularidad**. Si en el código intervino algún tercero (socio, freelance, agencia), definirlo ahora: corregir la titularidad después del depósito es considerablemente más difícil que declararla bien la primera vez.

2. Las funcionalidades originales

Las siete fichas que siguen son las que concentran decisiones de diseño propias. El resto de la plataforma (checkout, panel de fotógrafo, blog, campañas de email) es funcionalmente convencional y se deposita como código sin ficha individual.

F-01 · Álbum oculto con revelado selectivo por selfie

Denominación comercial propuesta: *Revelado por Selfie*

Objetivo. Permitir que un álbum sea públicamente accesible sin que ninguna fotografía sea visible, y que cada visitante vea únicamente las fotografías en las que aparece él mismo.

Actores. Visitante (autenticado o anónimo), fotógrafo titular del álbum, administrador.

Precondición. `Album.hiddenPhotosEnabled = true`.

Flujo.

1. El visitante entra al álbum. La grilla se sirve vacía: sin grant vigente, `filterPublicAlbumPhotosForHiddenVisitor` devuelve una lista de cero fotos. No se entregan miniaturas, ni URLs, ni conteos por foto.

2. El visitante sube una selfie.
3. **Validación de rostro único.** `detectFaceCount` cuenta rostros en la imagen. Cero → resultado `NO_FACE`. Más de uno → `MULTIPLE_FACES`. Ambos casos se rechazan.
4. **Búsqueda.** `searchFacesByImage` consulta la colección de Rekognition con `FaceMatchThreshold: 70` y `MaxFaces: 20`.
5. **Emisión del grant.** Si hay coincidencias, se crea un `HiddenAlbumGrant` cuyo campo `allowedPhotoIds` contiene la lista literal de identificadores de las fotografías donde apareció ese rostro, y `allowedCount` su cantidad.
6. Se entrega al visitante una cookie firmada (ver F-02) y la grilla se re-filtra: `photos.filter(p => allowed.has(p.id))`.
7. Cada acceso posterior a una fotografía individual revalida contra el grant: `validateHiddenAlbumPhotoAccess` exige que el `photoId` pertenezca a `allowedPhotoIds`. Un identificador adivinado no abre nada.

Decisión de diseño distintiva. El grant **no abre el álbum**: abre un subconjunto nominal. Las fotografías de terceros permanecen invisibles para siempre, incluso para alguien que ya se verificó correctamente. La alternativa obvia —verificar identidad y luego mostrar el álbum completo— fue descartada. Esa es la elección que hace propia a esta implementación.

Efecto colateral gestionado. Activar el modo oculto encola automáticamente el análisis facial de todas las fotos del álbum que aún no lo tengan (`ensureAlbumPhotosQueuedForHiddenMode`), creando registros `PhotoAnalysisJob` en estado `PENDING`. Sin esto, el primer visitante encontraría un álbum que no puede reconocerlo todavía.

Modelo de datos.

```
model Album {
  hiddenPhotosEnabled      Boolean @default(false)
  hiddenSelfieRetentionDays Int?
  scanProtectionEnabled    Boolean @default(true)
  enableFaceBulkPurchase   Boolean @default(false)
  faceBulkPriceCents       Int?
}
```

Archivos que la implementan. `app/api/albums/[id]/hidden/selfie/route.ts` · `app/api/albums/[id]/hidden/check-grant/route.ts` · `app/api/albums/[id]/search/face/route.ts` · `lib/hidden-album/filter-public-album-photos.ts` · `lib/hidden-album/prepare-hidden-album-photos.ts` · `lib/hidden-album-audit.ts` · `lib/faces/rekognition.ts`

[VERIFICAR] Los tres endpoints bajo `app/api/albums/[id]/` no pudieron leerse en esta pasada por un límite de profundidad de carpetas. La descripción del flujo se reconstruyó a partir del modelo de datos, del módulo de auditoría y del cliente de Rekognition, que son consistentes entre sí. Conviene contrastarla contra el código antes de firmar el depósito.

F-02 · Grant criptográfico de alcance fotográfico limitado

Objetivo. Sostener la autorización de F-01 sin consultar la base de datos en cada render y sin que el cliente pueda alterar su propio alcance.

Mecanismo.

```
payload = grantId : albumId : expiresAt
firma   = HMAC-SHA256(payload, HIDDEN_ALBUM_GRANT_SECRET)
cookie  = base64url( payload : firma )
```

La cookie `hidden_album_grant` se emite `HttpOnly`, `SameSite=Lax`.

Validación en cadena — cinco condiciones, todas obligatorias.

1. La firma HMAC recalculada coincide con la recibida.
2. `expiresAt` no venció (se comprueba antes de tocar la base).
3. El `albumId` del token es el álbum que se está pidiendo.
4. El registro `HiddenAlbumGrant` existe, no está revocado (`isRevoked`) y no venció en la base.
5. El `photoId` solicitado está incluido en `allowedPhotoIds`.

Decisión de diseño distintiva. La verificación es **doble y ordenada**: primero criptográfica y sin costo de base de datos, después contra el registro persistido. Un token perfectamente firmado sigue sin servir si el grant fue revocado. Y el alcance —qué fotos exactamente— vive del lado servidor, nunca en el token. Un atacante que reconstruyera el formato de la cookie no podría ampliar su propio alcance.

Identidad del visitante anónimo. Cookie `hidden_album_guest_id` con un UUID v4, un año de vigencia, validada por expresión regular al leerse. Permite correlacionar intentos de una misma persona sin exigirle registro.

Archivo. `lib/hidden-album-audit.ts`

F-03 · Auditoría de intentos con minimización de datos

Objetivo. Registrar cada intento de revelado con detalle suficiente para detectar abuso, sin acumular datos personales innecesarios.

Qué se guarda por intento (`HiddenAlbumAttempt`).

Campo	Contenido	Decisión
<code>ipHash</code>	<code>sha256(ip + sal)</code>	La IP nunca se persiste en crudo
<code>userAgent</code>	Truncado a 1024 caracteres	Límite duro en el tipo de columna
<code>deviceType</code>	<code>MOBILE</code> / <code>DESKTOP</code> / <code>UNKNOWN</code>	Derivado del user-agent, no del cliente

Campo	Contenido	Decisión
result	NO_FACE , MULTIPLE_FACES , MATCH_FOUND , NO_MATCH , EXPIRED_SESSION , RATE_LIMITED , ERROR	Taxonomía cerrada de siete resultados
facesInSelfieCount	Rostros detectados en la selfie	
bestMatchConfidence	Similitud del mejor match	
matchedFacesCount , photosMatchedCount , photosNoFaceCount , photosVisibleTotal	Métricas del universo evaluado	Permiten auditar el resultado sin volver a correr el reconocimiento
durationMs	Duración del intento	
selfieStored , selfieObjectKey , selfieExpiresAt	Estado del archivo de la selfie	Ver F-04
qrSessionId	Sesión de acceso por QR, si corresponde	

Decisión de diseño distintiva. El registro conserva **las métricas del intento pero no la evidencia**. Después de la caducidad queda constancia auditable de que alguien intentó, cuándo, desde qué tipo de dispositivo, con qué resultado y con qué confianza — y ya no existe la selfie con la que lo hizo. Es lo contrario de la práctica habitual, que guarda el archivo y descarta las métricas.

Superficies administrativas. /admin/auditoria-selfies · /admin/hidden-album-attempts · /admin/antifraude

Archivos. lib/hidden-album-audit.ts (hashIp , getDeviceType , truncateUserAgent , getClientIp , getOrCreateGuestId)

F-04 · Ciclo de vida biométrico con borrado selectivo

Objetivo. Que el dato biométrico caduque solo, y que su eliminación no destruya la relación comercial con la persona.

Dos relojes independientes.

Reloj corto — la selfie del intento. hidden-album-cleanup recorre los HiddenAlbumAttempt con selfieExpiresAt vencido, borra el objeto de R2 y pone selfieStored=false, selfieObjectKey=null, selfieExpiresAt=null. El registro del intento sobrevive intacto. La retención es configurable por álbum (Album.hiddenSelfieRetentionDays). Procesa de a 50 registros por corrida.

Reloj largo — la plantilla facial. biometric-cleanup recorre los AlbumInterest con expiresAt vencido y biometricDeletedAt nulo, y por cada uno: elimina el rostro de la colección de Rekognition (deleteFace), borra la selfie de R2, y marca en base faceId=null, selfieKey=null, biometricDeletedAt=now. Procesa hasta 100 por corrida.

Decisión de diseño distintiva — el borrado es selectivo. El código lo deja escrito como comentario en el propio archivo:

```
// NO eliminar: email, whatsapp, name, etc.
```

Caduca lo biométrico. El email, el WhatsApp y el nombre permanecen. Una persona que dio su selfie hace más de noventa días sigue siendo un contacto comercial válido y sigue recibiendo la secuencia de avisos, pero ya no existe su plantilla facial en ningún sistema. Separar el dato sensible del dato de contacto, en vez de borrar el registro completo, es una decisión de producto deliberada.

Tolerancia a fallos. Si Rekognition o R2 fallan al borrar, la base se actualiza igual: el registro queda marcado como eliminado antes que quedar en un limbo donde el dato figura vigente pero ya no está. El error se registra pero no aborta el lote.

Consentimiento y revocación. Página dedicada `/consentimiento-biometrico`; campos `biometricConsent` y `biometricConsentAt` en `AlbumInterest`; endpoints `/api/users/me/face-consent` y `/api/users/me/revoke-face-consent`; borrado a pedido vía `deleteFaceTemplatesForUser`; toda operación deja rastro en `PrivacyEvent`. Los enlaces de borrado que viajan por email llevan token HMAC-SHA256 con vencimiento.

Archivos. `app/api/cron/hidden-album-cleanup/route.ts` · `app/api/cron/biometric-cleanup/route.ts` · `lib/privacy-face.ts` · `lib/legal/biometric-consent-content.ts` · `app/consentimiento-biometrico/page.tsx`

F-05 · Ventana de escaneo sobre fotografía no comprada

Denominación comercial propuesta: *Ventana de Escaneo*

Objetivo. Permitir que el comprador evalúe una fotografía completa sin poder capturarla nítida.

Mecanismo. La imagen ampliada se muestra desenfocada, y una franja horizontal nítida la recorre de arriba hacia abajo en ciclo continuo. En cualquier instante hay una sección legible y el resto no lo está.

Parámetros — cada uno es una decisión.

Parámetro	Valor	Razón
Alto de la franja	<code>clamp(8rem, 22vh, 13rem)</code>	Aproximadamente 6 cm en pantalla típica; 128 px en móvil, 208 px en escritorio
Desenfoque de base	<code>clamp(6px, 1.2vh, 11px)</code>	Impide leer detalle sin ocultar la composición
Velocidad	128 px/s	Constante: el recorrido se siente igual en fotos verticales, horizontales y cuadradas
Velocidad con <code>prefers-reduced-motion</code>	33 px/s	Sigue recorriendo , mucho más lento

Parámetro	Valor	Razón
Duración del ciclo	Acotada entre 2.700 y 24.000 ms	Ni frustrante en fotos chicas ni interminable en fotos grandes
Recorrido	<code>frameHeight + bandHeight</code>	La franja entra por arriba y sale por abajo, no aparece ni desaparece dentro del cuadro

Decisión de diseño distintiva — la velocidad es constante, no la duración. Lo intuitivo sería fijar un ciclo de N segundos para todas las fotos; eso haría que una foto vertical se escanee visiblemente más rápido que una apaisada. Acá se fija la velocidad y se deriva la duración, con topes en ambos extremos.

Segunda decisión — el rectángulo contenido. `computeContainedRect` calcula el área que la fotografía realmente ocupa dentro de su caja bajo `object-fit: contain`, con sus desplazamientos. Sin ese cálculo la franja se pintaría también sobre las bandas vacías laterales o superiores, delatando el truco y quedando visualmente sucio.

Tercera decisión — accesibilidad sin renunciar a la protección. Ante `prefers-reduced-motion` la respuesta habitual sería detener la animación; eso dejaría la foto nítida en una franja fija o desenfocada entera. La implementación **reduce la velocidad a un cuarto** en lugar de detenerla.

Regla de aplicación. `shouldApplyScanProtection({enabled, purchased})` devuelve verdadero solo si la vista lo pide y la foto **no** fue comprada. Comprada, se ve limpia. Controlado por álbum vía `Album.scanProtectionEnabled` (activo por defecto).

Archivos. `lib/photo/scan-protection.ts` · `lib/photo/scan-protection.test.ts` (lógica pura, sin React ni DOM, deliberadamente testeable)

F-06 · Lupa sobre vista previa protegida, sin costo de servidor

Objetivo. Dejar que el comprador inspeccione detalle —si su hijo salió con los ojos abiertos— sin entregarle nunca bytes del archivo original.

Arquitectura de tres capas sobre una sola URL.

1. Base: `` con `filter: blur(28px)` y `scale(1.04)`.
2. Lupa circular: `background-image` **con la misma URL**, zoom 1,5×, seguimiento del puntero por `requestAnimationFrame` + `transform`.
3. Refuerzo de marca de agua en CSS (patrón diagonal + texto), `pointer-events: none`.

Decisión de diseño distintiva — cero procesamiento adicional. La lupa reutiliza la imagen que el navegador ya tiene en caché. No se agregaron trabajos de Sharp, ni workers, ni crons, ni claves nuevas en R2, ni migraciones, ni endpoints, ni tiles, ni recortes. Mover la lupa genera **cero requests**. Abrir el modal genera el mismo request que generaba antes.

Escenario	Requests
Abrir el visor (caché normal)	1 GET <code>mode=preview</code>
Con caché deshabilitada	Hasta 2 GET de la misma URL — nunca del original

Escenario	Requests
Mover la lupa	0

Cuarta decisión — el límite es intencional. La lupa amplía sobre una previsualización de ~640 px de lado. El detalle disponible es limitado *a propósito*: la protección no viene de ocultar la lupa sino de que no haya nada mejor que ampliar. Está documentado como limitación asumida, no como defecto pendiente.

Adaptación táctil. En escritorio la lupa se activa al pasar el puntero. En móvil hay un botón explícito "Activar lupa", porque activarla al tocar rompería los gestos de anterior / siguiente / cerrar. La lupa es `aria-hidden`; los controles del visor quedan intactos.

Archivos. `components/photo/ProtectedPhotoMagnifier.tsx` · `lib/images/contained-image-metrics.ts` · integrado en `PhotoSlideViewer` mediante la prop `enableProtectedMagnifier`, activo en `ClientAlbumView`, `EventGalleryGrid` y `ComprarClient`.

F-07 · Marca de agua trazable por visor

Objetivo. Que una fotografía filtrada identifique a quien la filtró.

Mecanismo. El texto embebido en la marca de agua se compone en el momento de servir la imagen, e incluye: número de álbum, número de orden (si existe), identidad del visor — email si está autenticado, identificador de visitante si no— y fecha y hora en formato local `es-AR`.

Album #142 - Orden #3871 - Usuario: cliente@ejemplo.com - 03/09/2026 14:22

Decisión de diseño distintiva. La mayoría de las marcas de agua identifican al *autor* de la foto. Ésta identifica además al *destinatario*. Una captura de pantalla que aparezca en un grupo de mensajería lleva escrito quién la tenía abierta y cuándo. Cambia la naturaleza de la protección: de disuasión genérica a atribución individual.

Implementación. Renderizada server-side con Sharp sobre SVG con tipografía embebida. Patrón mosaico en múltiples rotaciones simultáneas, más un texto central de mayor tamaño. Dos modos, `standard` y `strong`, con opacidades y calidad JPEG diferenciadas (72 y 66 respectivamente; 38 para miniaturas). Lado máximo 1200 px. Trazo negro bajo relleno blanco (`paint-order="stroke fill"`) para que sobreviva sobre fondos claros y oscuros.

Versionado de caché. La constante `PROTECTED_PREVIEW_WATERMARK_VERSION` (actualmente `"ppw3"`) forma parte de la clave de caché en R2: cambiar la marca invalida los derivados sin borrado manual.

Caché diferenciada. Variantes pregeneradas: `public`, `max-age=31536000`, `immutable`. Variantes dinámicas por visor: `private`, `no-store`.

Archivos. `lib/watermarking.ts` · `lib/images/protected-preview-watermark.ts` · `lib/images/watermark-svg-font.ts` · `lib/images/watermark-photographer-center.ts` · `lib/images/watermark-render.ts`

3. Funcionalidades secundarias con elementos propios

Se depositan como código y se describen brevemente; no llevan ficha completa.

Compra masiva por rostro. `Album.enableFaceBulkPurchase` + `faceBulkPriceCents` habilitan comprar de una vez todas las fotos donde aparece una persona, a precio de paquete. El reconocimiento facial deja de ser solo un buscador y pasa a ser una unidad de venta. `lib/face-bulk-offer.ts`.

Preventa canjeable escolar con máquina de estados por selfie. Seis fases de experiencia (`needs_upload`, `received`, `searching`, `no_matches`, `photos_ready`, `can_redeem`) derivadas de la combinación entre selfies cargadas, estados de los ítems del pedido y disponibilidad de fotos. Cada fase tiene su propio texto en pantalla. Permite cobrar antes de que existan las fotos y resolver la identificación después. `lib/preventa-canjeable/preventa-selfie-state.ts`.

Denuncias de propiedad intelectual con ocultamiento trazable. Modelos `ContentReport` y `ContentReportEvent`. El ocultamiento preventivo marca `Photo.isRemoved` con `removedReason = content_report:{id}:temporary|removed`, y la restauración **solo procede si la razón pertenece a esa denuncia** — de modo que levantar una denuncia no puede reactivar por accidente una foto retirada por derecho de imagen. Los archivos de R2 nunca se borran automáticamente. La IP del denunciante se guarda minimizada. `docs/legal/LEGAL_IP_CONTENT_REPORTS_ETAPA01.md`.

Registro de intentos de captura de pantalla. `app/api/a/[id]/screenshot-log/route.ts`. [VERIFICAR] — endpoint no leído en esta pasada.

4. Qué vía protege qué

Funcionalidad	Depósito DNDA	Marca INPI	Secreto comercial
F-01 Revelado por Selfie	Código, flujo, esquema de datos	Sí — prioritaria	Umbral 70 y política de alcance (server-side)
F-02 Grant criptográfico	Código y protocolo	—	Esquema de firma y secretos de entorno
F-03 Auditoría minimizada	Código y esquema	—	Sal de hasheo
F-04 Ciclo biométrico	Código y política	—	Ventanas de retención
F-05 Ventana de Escaneo	Código y constantes	Sí — prioritaria	⚠ Débil: las constantes viajan al navegador
F-06 Lupa protegida	Código y composición	Posible	⚠ Débil: es código de cliente
F-07 Marca de agua trazable	Código y composición del texto	Posible	Parámetros de render (server-side)

Advertencia sobre secreto comercial. F-05 y F-06 se ejecutan en el navegador del usuario: cualquiera puede abrir las herramientas de desarrollo y leer los valores. Para esas

dos, el secreto comercial no es una vía real y toda la protección descansa en el depósito de autoría y en la marca. Conviene no construir la estrategia sobre una confidencialidad que técnicamente no existe.

5. Marcas a registrar en INPI

Ésta es la única vía que otorga un derecho de exclusión efectivo. Nadie podrá llamar a su funcionalidad como se llame la tuya.

Prioridad 1 — denominativas

Signo	Clases sugeridas
ComprameLaFoto	9, 35, 42 (y 40 si hay impresión propia)
DNX Suite	9, 42
Revelado por Selfie	9, 42
Ventana de Escaneo	9, 42

Prioridad 2 — a evaluar

Lupa Protegida · Marca Testigo (o el nombre que se elija para F-07) · Compra por Rostro

Notas.

- Clase 9: software. Clase 42: servicios de software como servicio. Clase 35: servicios de marketplace y publicidad. Clase 40: tratamiento de materiales, incluye revelado e impresión fotográfica. Cada clase se abona por separado.
- Antes de presentar conviene una búsqueda de antecedentes en la base del INPI: nombres descriptivos pueden ser objetados por falta de distintividad. "Ventana de Escaneo" es más vulnerable en ese sentido que un nombre de fantasía.
- El logotipo se registra aparte, como marca mixta.

6. Contenido del soporte a depositar

/dossier/ DOSSIER-DNDA-COMPRAMELAFOTO.pdf LEEME.txt	Este documento Identificación de obra y titular
/codigo-fuente/ apps/compramelafoto/ packages/db/prisma/schema.prisma packages/	Sin node_modules, .next, .turbo ni secretos 455 modelos Paquetes compartidos utilizados
/anexos/ capturas/ marca de agua diagramas/ dependencias.txt MANIFIESTO.txt	Interfaz: álbum oculto, selfie, escaneo, lupa, Estados del grant; ciclo de vida biométrico Terceros y sus licencias SHA-256 de cada archivo incluido

Sobre el manifiesto. El listado de hashes es lo que permite demostrar, años después, que el contenido depositado es exactamente el que se está exhibiendo. Se genera al armar el paquete y se imprime también en papel dentro del sobre.

Sobre las dependencias. El anexo de licencias de terceros cumple una función defensiva: deja explícito qué partes del sistema **no** son obra propia. Un dossier que reclama autoría sobre todo, incluidos los paquetes de código abierto, se debilita entero.

7. Checklist del trámite

- ☐ Definir autores y porcentajes de titularidad
 - ☐ Decidir alcance: ¿solo ComprameLaFoto o DNX Suite completa? (mismo arancel)
 - ☐ Verificar los tres archivos marcados [VERIFICAR] en este documento
 - ☐ Tomar las capturas de pantalla de los anexos
 - ☐ Generar el snapshot del código y el manifiesto de hashes
 - ☐ Exportar este dossier a PDF
 - ☐ Grabar en CD, DVD o pendrive
 - ☐ Abonar el arancel: \$1.400 al fondo cooperador
 - ☐ Iniciar el trámite en TAD (Trámites a Distancia)
 - ☐ Sobre de papel madera cerrado, tamaño A4 mínimo, firmado sobre los puntos de cierre
 - ☐ Presentar: Moreno 1230, CABA (lunes a viernes 9:30-14:30), por correo a Moreno 1228 C1091AAZ, o receptoría del interior
 - ☐ **Agendar la renovación:** vence a los 3 años; se renueva dentro de los 30 días anteriores al vencimiento
 - ☐ En paralelo: búsqueda de antecedentes e inicio de las marcas prioritarias en INPI
-

Fuentes

- [Depósito en custodia de obra inédita software — DNDA](#)
- [Obras inéditas — Dirección Nacional del Derecho de Autor](#)
- [Ley 24.481 de Patentes de Invención y Modelos de Utilidad — arts. 4 y 6](#)